基于CO2排放量数据集的数据分析及可视化

https://dblab.xmu.edu.cn/blog/2738/

一、实验环境

Linux：Ubuntu18.04
Hadoop：3.1.3
Spark：3.2.0
Python3.7
Scala：2.12.15
Jupyter notebook

二、数据预处理

2.1 数据集简介

本次实验采用的数据集是全球二氧化碳排放量数据集，是从Kaggle上下载的（点击这里从百度网盘下载数据集）。该数据集提供了全球化石二氧化碳排放量的国家级调查，包括总排放量、煤炭、石油、天然气、水泥、燃除和其他来源的排放量以及人均排放量。对于希望按国家量化全球二氧化碳排放水平并了解这些排放来源的研究人员来说，这个数据集可能是一个宝贵的资源。该数据集包括了从1750年至2021年的各个国家的数据。该数据集的各个字段说明如下：

字段	说明	举例
Country	国家/地区的名称	Chain
ISO 3166-1 alpha-3	国家/地区的三个字母代码	CHA
Year	数据的年份	2021
Total	指定年份国家/地区的二氧化碳排放总量	37123.85035
Coal	指定年份该国煤炭的二氧化碳排放量	14979.6
Oil	指定年份该国石油的二氧化碳排放量	11837.16
Gas	指定年份该国天然气的二氧化碳排放量	7921.829
Cement	指定年份该国水泥生产的二氧化碳排放量	1672.592
Flaring	指定年份该国燃除作业的二氧化碳排放量	416.5256
Other	指定年份该国其他来源的二氧化碳排放量	296.1457
Per Captia	指定年份国家/地区的人均二氧化碳排放量	4.693699

2.2 数据集清洗

本文使用pandas读取数据并且进行数据清洗和预处理。具体步骤如下：

1.首先国家的ISO 3166-1 alpha-3代码对本文的分析没有多大用处，就过滤掉该字段。

2.由于该数据是从1750年开始收集的，由于种种原因，总有一些国家的某些年数据是丢失的。因此，过滤掉某个国家某年数据全为空的行。

3.该数据集中存在“global 和 International Transport”。这两个字段表示全球和国际排放量。特别地，这两个数据不代表国家，因此过滤掉这两个数据。

处理数据集的python代码和处理后的数据集展示如下：

```
1.# 数据处理文件，处理数据集
2.import pandas as pd
3.data = pd.read_csv('../GCB2022v27_MtCO2_flat.csv')
4.# print(data.head(100))
5.# 去除字段国家简称字段
```

```python
6.# 过滤掉total = 0的字段,填充nan到0
7.# 去除global 和 International Transport的数据
8.data = data.fillna(0)
9.with open('dataset.csv','w+',encoding='utf-8') as f:
10.    for line in data.values:
11.        if line[3] == 0.0:
12.            continue
13.        if line[0] == "Global" or line[0] == "International Transport":
14.            continue
15.        f.write(str(line[0])+'\t'+str(line[2])+'\t'
16.                +str(line[3])+'\t'+str(line[4])+'\t'+str(line[5])+'\t'+
17.                str(line[6])+'\t'+str(line[7])+'\t'+str(line[8])+'\t'+
18.                str(line[9])+'\t'+str(line[10])+'\n')
```

language-python

处理后的部分数据展示如下：

China	2012	9779.282	7464.806	1176.712	277.5993	714.7817	3.659815	141.7225	7.156126
China	2013	9956.308	7493.454	1234.622	320.1275	748.3231	3.564518	156.2169	7.235162
China	2014	9998.621	7425.067	1255.605	370.2216	778.6271	3.877594	165.2232	7.218233
China	2015	9866.904	7266.979	1328.805	380.0522	721.995	3.854226	165.219	7.079569
China	2016	9764.98	7071.404	1357.838	421.7262	743.0441	3.04058	167.9275	6.965584
China	2017	10011.11	7163.318	1430.587	486.3594	758.1852	2.924697	169.7332	7.098687
China	2018	10353.88	7316.4	1500.474	575.3689	786.7446	3.350856	171.5389	7.306542
China	2019	10741	7543.157	1559.728	630.1714	826.876	3.806248	177.2568	7.554165
China	2020	10956.21	7605.717	1627.725	687.8809	858.2326	5.118808	171.5389	7.68895
China	2021	11472.37	7955.985	1713.34	773.8661	852.9613	4.677478	171.5389	8.045741

2.3 启动HDFS和Spark

在开始作业之前，需要先在虚拟机上（已经配置好环境）开启hdfs和spark。

```
xmw@031802433:~$ cd /usr/local/hadoop-3.1.3/etc/hadoop/
xmw@031802433:/usr/local/hadoop-3.1.3/etc/hadoop$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [031802433]
xmw@031802433:/usr/local/hadoop-3.1.3/etc/hadoop$ cd /usr/local/spark-3.2.0/
xmw@031802433:/usr/local/spark-3.2.0$ ./sbin/start-all.sh
starting org.apache.spark.deploy.master.Master, logging to /usr/local/spark-3.2.
0/logs/spark-xmw-org.apache.spark.deploy.master.Master-1-031802433.out
localhost: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local
/spark-3.2.0/logs/spark-xmw-org.apache.spark.deploy.worker.Worker-1-031802433.ou
t
```

2.4 上传数据到HDFS

首先在HDFS上创建一个目录'/user/xmw/'，如下所示：

```
xmw@031802433:~$ hdfs dfs -mkdir -p /user/xmw/
xmw@031802433:~$ hdfs dfs -ls .
xmw@031802433:~$ hdfs dfs -ls
xmw@031802433:~$ hdfs dfs -ls /user/
Found 1 items
drwxr-xr-x   - xmw supergroup          0 2023-05-15 13:51 /user/xmw
xmw@031802433:~$ hdfs dfs -ls /user/xmw
```

把处理好的数据(dataset.csv)上传到HDFS，如下所示：

```
xmw@031802433:~$ hdfs dfs -ls /user/xmw
xmw@031802433:~$ hdfs dfs -put /home/xmw/dataset.csv /user/xmw/
2023-05-15 13:54:56,256 INFO sasl.SaslDataTransferClient: SASL encryption trust
check: localHostTrusted = false, remoteHostTrusted = false
xmw@031802433:~$
```

通过ls命令查看是否上传成功，如下所示：

```
xmw@031802433:~$ hdfs dfs -ls /user/xmw
Found 1 items
-rw-r--r--   1 xmw supergroup    1794175 2023-05-15 13:54 /user/xmw/dataset.csv
xmw@031802433:~$
```

三、使用 Spark 将数据转为 DataFrame

由于读入的文件是 csv 文件，是结构化的数据，因此可以将数据创建为 DataFrame 方便进行分析。为了创建 DataFrame，首先需要将

HDFS 上的数据加载成 RDD，再将 RDD 转化为 DataFrame。下面代码段完成从文件到 RDD 再到 DataFrame 的转化：

```
1.import findspark
2.findspark.init()
3.from pyspark import SparkContext
4.from pyspark.sql import SparkSession, Row
5.from pyspark.sql.functions import col, max
6.from pyspark.sql.types import StringType, StructField, StructType,IntegerType,FloatType
7.import json
8.import csv
9.import os
10.from pyspark.sql import functions as F
11.sc = SparkContext('local', 'spark_project')
12.sc.setLogLevel('WARN')
13.spark = SparkSession.builder.getOrCreate()
14.
15.schemaString = "Country,Year,Total,Coal,Oil,Gas,Cement,Flaring,Other,PerCapita"
16.fields = []
17.for field in schemaString.split(","):
18.    if field == 'Country':
19.        a = StructField(field, StringType(), True)
20.    elif field == 'Year':
21.        a = StructField(field,IntegerType(),True)
22.    else:
23.        a = StructField(field,FloatType(), True)
24.    fields.append(a)
25.# fields.append(a)
26.schema = StructType(fields)
27.CO2Rdd1 = sc.textFile('/user/xmw/dataset.csv')
```

```
28.28.CO2Rdd = CO2Rdd1.map(lambda x:x.split("\t")).map(lambda p: Row(p[0],int(p[1]),float(p[2]),float(p[3]),float(p[4]),float(p[5]),float(p[6]),float(p[7]),float(p[8]),float(p[9])))
29.29.mdf = spark.createDataFrame(CO2Rdd, schema)
30.30.mdf.createOrReplaceTempView("usInfo")
31.31.mdf.show()
32.32.# 展示数据表结构
33.33.mdf.printSchema()
```

language-python

通过show()函数查看已经转化成DataFrame类型的数据，这里只展示了部分结果，如下所示：

```
/home/xmw/anaconda3/bin/python /home/xmw/桌面/code/analyst.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
2023-05-19 11:50:54,761 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes wher
+-----------+----+------------------+--------------------+--------------------+--------+--------------------+-------+-----+--------------------
|    Country|Year|             Total|                Coal|                 Oil|     Gas|              Cement|Flaring|Other|              PerCap
+-----------+----+------------------+--------------------+--------------------+--------+--------------------+-------+-----+--------------------
|Afghanistan|1949|          0.014656|            0.014656|                 0.0|     0.0|                 0.0|    0.0|  0.0|
|Afghanistan|1950|          0.084272|            0.021068|            0.063204|     0.0|                 0.0|    0.0|  0.0|               0.011
|Afghanistan|1951|            0.0916|            0.025648|            0.065952|     0.0|                 0.0|    0.0|  0.0|0.012098000000000
|Afghanistan|1952|            0.0916|            0.031708|0.059891999999999994|     0.0|                 0.0|    0.0|  0.0|               0.011
|Afghanistan|1953|0.10625599999999999|0.037949000000000004|            0.068307|     0.0|                 0.0|    0.0|  0.0|               0.013
|Afghanistan|1954|0.10625599999999999|            0.042502| 0.06375399999999999|     0.0|                 0.0|    0.0|  0.0|0.013511000000000
|Afghanistan|1955|0.15388800000000002|0.062287999999999996|              0.0916|     0.0|                 0.0|    0.0|  0.0|0.019303999999999
|Afghanistan|1956|            0.1832|0.062287999999999996| 0.12091199999999999|     0.0|                 0.0|    0.0|  0.0|0.022652000000000
|Afghanistan|1957|           0.29312| 0.07694400000000001| 0.21617600000000003|     0.0|                 0.0|    0.0|  0.0|               0.035
|Afghanistan|1958|           0.32976|              0.0916| 0.23815999999999998|     0.0|                 0.0|    0.0|  0.0|               0.039
|Afghanistan|1959|          0.384571|             0.10992| 0.25648000000000004|     0.0|            0.018171|    0.0|  0.0|0.045413999999999
|Afghanistan|1960|0.41388500000000006|            0.127115|            0.268758|     0.0|            0.018012|    0.0|  0.0|               0.048
|Afghanistan|1961|0.49079799999999996|            0.175872|             0.29312|     0.0|            0.021806|    0.0|  0.0|               0.055
|Afghanistan|1962| 0.6885939999999999|            0.296784|            0.362736|     0.0|0.029074000000000003|    0.0|  0.0|               0.076
```

使用printSchema()函数查看表结构，可以看出一共有10个字段，包括了每个字段的类型，如下所示：

```
root
 |-- Country: string (nullable = true)
 |-- Year: integer (nullable = true)
 |-- Total: float (nullable = true)
 |-- Coal: float (nullable = true)
 |-- Oil: float (nullable = true)
 |-- Gas: float (nullable = true)
 |-- Cement: float (nullable = true)
 |-- Flaring: float (nullable = true)
 |-- Other: float (nullable = true)
 |-- PerCapita: float (nullable = true)
```

四、使用 Spark 对 HDFS 中的数据进行分析

本小节使用python语言，使用了Spark Core、Spark SQL、Spark MLlib组件，分析结果也可以保存到csv文件中。

1. 查询2021年中国的排放数据

```python
1.# 1.查询2021年中国的排放总量
2.df1 = spark.sql(
3.    "SELECT * FROM usInfo WHERE Country = 'China' AND Year = '2021'")
4.# 将结果保存为文本文件
5.df1.repartition(1).write.csv('/user/xmw/df1',mode="overwrite")
6.df1.show()
```

language-python

```
+-------+----+---------+---------+-------+--------+---------+--------+---------+---------+
|Country|Year|    Total|     Coal|    Oil|     Gas|   Cement| Flaring|    Other|PerCapita|
+-------+----+---------+---------+-------+--------+---------+--------+---------+---------+
|  China|2021|11472.369|7955.9854|1713.34|773.8661|852.96136|4.677478|171.53888| 8.045741|
+-------+----+---------+---------+-------+--------+---------+--------+---------+---------+
```

从上面的分析结果可以看出，2021年中国的总排放量为11472.369，人均排放量为8.045等。

2. 统计总CO2排放量

```python
1.# 2.统计总能源使用量
2.df2 = mdf.selectExpr("SUM(Total) AS total_energy")
3.df2.show()
4.df2.repartition(1).write.csv('/user/xmw/df2',mode="overwrite")
```

language-python

```
+------------------+
|      total_energy|
+------------------+
|1693867.3560400738|
+------------------+
```

3. 计算每个国家的平均人均能源使用量

```python
1.# 3.计算每个国家的平均人均能源使用量(总的)，不是每一年的
2.df3 = mdf.groupBy("Country").agg({"PerCapita": "avg"}).withColumnRenamed("avg(PerCapita)", "AvgPerCapita")
3.df3.show()
```

```
4. 4.df3.repartition(1).write.csv('/user/xmw/df3',mode="overwrite")
```
language-python

Country	AvgPerCapita
Côte d'Ivoire	0.4103619224624708 3
Chad	0.06135427016055300 4
Micronesia (Feder...	1.346740065018335 9
Anguilla	8.991637319326 4
Paraguay	0.5398832949964 95
Russia	4.7235792580772 85
Macao	2.0260979159570 79
Yemen	0.5923172905265 043
Senegal	0.4331148589844 8154
Sweden	3.5157297991783 074
Kiribati	0.4206226729467 267
Guyana	1.9942815370029 874
Eritrea	0.1960509108269 9158
Philippines	0.4504396758441 414
Djibouti	0.634266362629003
Occupied Palestin...	0.5225029997527 599
Tonga	0.6318808713306 984

在原来的数据集中，有一个字段表示某个国家某一年的人均CO2排放量，但是没有统计某个国家在历史上的人均CO2排放量。因此，这里

通过统计每个国家每年的人均CO2排放量，来得出某个国家1750-2021年的人均CO2排放量。

4. 获取排放总量最高的国家和对应的年份

```python
1.#4.获取排放总量最高的国家和对应的年份
2.df4 = mdf.select("Country", "Year", "Total").orderBy(F.col("Total").desc())
3.df4.show(1)
4.df4.repartition(1).write.csv('/user/xmw/df4',mode="overwrite")
5.print("排放总量最高的国家:", df4.first()["Country"])
6.print("对应的年份:", df4.first()["Year"])
```

language-python

```
+-------+----+----------+
|Country|Year|     Total|
+-------+----+----------+
|  China|2021|11472.369|
+-------+----+----------+
only showing top 1 row

排放总量最高的国家: China
对应的年份: 2021
```

从上面的分析可以看出，在数据集中，2021年的中国CO2排放量是最高的。

5. 2021年人均排放最高的国家

```python
#5.2021年人均排放最高的国家
df_2021 = mdf.filter(mdf.Year == 2021)
# 按人均排放降序排序并获取第一条记录
df5 = df_2021.orderBy(F.col("PerCapita").desc())
df5.show(1)
df5.repartition(1).write.csv('/user/xmw/df5',mode="overwrite")
# 输出人均排放最高的国家和对应的人均排放值
country = df5.first()["Country"]
per_capita = df5.first()["PerCapita"]
print("2021年人均排放最高的国家:", country)
print("人均排放值:", per_capita)
```

language-python

```
+-------+----+--------+----+-------+--------+--------+--------+-----+---------+
|Country|Year|   Total|Coal|    Oil|     Gas|  Cement| Flaring|Other|PerCapita|
+-------+----+--------+----+-------+--------+--------+--------+-----+---------+
|  Qatar|2021|95.66718| 0.0|8.67304|83.53126|1.571676|1.891205|  0.0|35.587357|
+-------+----+--------+----+-------+--------+--------+--------+-----+---------+
only showing top 1 row

2021年人均排放最高的国家: Qatar
人均排放值: 35.58735656738281
```

从上面分析结果可以看出2021年人均CO2排放量最高的国家是卡塔尔。我们可以看到，2023年世界杯在卡塔尔举办，卡塔尔需要进行大量的基建。加上卡塔尔人口较少，因此人均排放量最高。

6. 中国历史排放总量（2000年后）

```python
#6. 中国历史排放总量（2000-2021）
# 过滤出中国2000年以后的数据
df6 = mdf.filter((mdf.Country == "China") & (mdf.Year >= 2000))
```

```
4. 4.df6.show()
5. 5.df6.repartition(1).write.csv('/user/xmw/df6',mode="overwrite")
```

language-python

```
+-------+----+---------+---------+---------+---------+---------+--------+---------+---------+
|Country|Year|    Total|     Coal|      Oil|      Gas|   Cement| Flaring|    Other|PerCapita|
+-------+----+---------+---------+---------+---------+---------+--------+---------+---------+
|  China|2000| 3644.464|2601.7368|648.68555| 59.79648| 244.3882|     0.0|89.857185| 2.883053|
|  China|2001|3723.7307| 2637.655| 659.3002|67.157455| 270.8282|     0.0|    88.79|  2.92576|
|  China|2002| 4112.459| 3018.634|  653.086|53.992702| 296.5903|     0.0|   90.156| 3.210536|
|  China|2003| 4827.446| 3601.712| 727.0988|60.745457|345.18613|     0.0| 92.70359| 3.745477|
|  China|2004| 5223.755|3835.2883|839.09265| 71.52128|379.99216|     0.0| 97.86024| 4.028136|
|  China|2005| 5876.555|  4424.39|850.17255| 84.95717|411.64877|     0.0| 105.3869| 4.503496|
|  China|2006|6488.8037| 4905.374| 898.5337|103.75349| 470.0866|     0.0| 111.0558| 4.941642|
|  China|2007| 6978.612| 5289.904| 927.3511|129.51508|514.98083|     0.0| 116.8613| 5.280773|
|  China|2008| 7496.832|5729.4736| 970.1173| 149.6744| 525.9245|     0.0| 121.6423| 5.636008|
|  China|2009|7886.5327|   6028.5| 982.3477|164.40369|583.56024|     0.0|  127.721| 5.889315|
|  China|2010| 8616.652|6568.7935|1092.5718| 199.5854| 639.5919|     0.0|   116.11| 6.391268|
|  China|2011| 9528.556| 7309.786|1123.8588|245.98997| 708.5643|     0.0| 140.3565| 7.021286|
|  China|2012| 9779.281|7464.8066|1176.7119| 277.5993| 714.7817|3.659815| 141.7225| 7.156126|
|  China|2013| 9956.309|7493.4546| 1234.622|320.12747| 748.3231|3.564518|156.21687| 7.235162|
|  China|2014| 9998.621| 7425.067|1255.6052|370.22156| 778.6271|3.877594|165.22316| 7.218233|
|  China|2015| 9866.904| 7266.979| 1328.805|380.05222|721.99493|3.854226|165.21902| 7.079569|
|  China|2016|9764.9795| 7071.404|1357.8375|421.72623|743.04407| 3.04058|167.92754| 6.965584|
```

上面统计了中国2000-2021年间的排放总量。可以看出，基本呈现递增趋势。

7. 查询每年排放总量最高的国家

```
1. 1.7. 查询每年排放总量最高的国家
2. 2.使用 Spark SQL 查询每年排放总量最高的国家
3. 3.query = """
4. 4.    SELECT Year, Country, Total
5. 5.    FROM (
6. 6.        SELECT Year, Country, Total, ROW_NUMBER() OVER (PARTITION BY Year ORDER BY Total DESC) as rank
7. 7.        FROM usInfo
8. 8.    ) ranked
9. 9.    WHERE rank = 1
10. 10.    ORDER BY Year
```

```python
11."""
12.df7 = spark.sql(query)
13.df7.repartition(1).write.csv('/user/xmw/df7',mode="overwrite")
14.# 显示所有查询结果
15.df7.show(300)
```
language-python

Year	Country	Total
1750	United Kingdom	9.350528
1751	United Kingdom	9.350528
1752	United Kingdom	9.354192
1753	United Kingdom	9.354192
1754	United Kingdom	9.357856
1755	United Kingdom	9.36152
1756	United Kingdom	10.006384
1757	United Kingdom	10.010048
1758	United Kingdom	10.013712
1759	United Kingdom	10.017376
1760	United Kingdom	10.017376
1761	United Kingdom	10.97368
1762	United Kingdom	10.977344
1763	United Kingdom	10.981008
1764	United Kingdom	10.984672
1765	United Kingdom	10.988336

1890	USA	401.74728
1891	USA	427.4246
1892	USA	450.44922
1893	USA	454.09485
1894	USA	425.05035
1895	USA	479.9481
1896	USA	480.88248
1897	USA	500.28714
1898	USA	545.3215
1899	USA	625.534
1900	USA	662.7384
1901	USA	721.5275
1902	USA	764.4957
1903	USA	894.4069
1904	USA	882.05756
1905	USA	986.4824
1906	USA	1035.4712
1907	USA	1203.8615

```
|2006|          China| 6488.8037|
|2007|          China|  6978.612|
|2008|          China|  7496.832|
|2009|          China| 7886.5327|
|2010|          China|  8616.652|
|2011|          China|  9528.556|
|2012|          China|  9779.281|
|2013|          China|  9956.309|
|2014|          China|  9998.621|
|2015|          China|  9866.904|
|2016|          China| 9764.9795|
|2017|          China| 10011.107|
|2018|          China| 10353.877|
|2019|          China| 10740.996|
|2020|          China| 10956.213|
|2021|          China| 11472.369|
+----+---------------+----------+
```

上面统计了每年CO2排放量最高的国家。从部分截图可以看出，在1750-1899年之间，总排放量最高的国家是英国；在1890-2005年之间，总排放量最高的国家是美国。2005年之后，总排放量最高的国家是中国。这个数据也基本和历史上国家之间的工业水平强弱相对应。第一次工业革命最早在英国发生，带动了英国的工业生产水平；第二次工业革命主要在美国和西欧，美国的崛起和英国的衰落使得美

国称为CO2排放量最高的国家；20世纪后，中国的崛起，使得中国成为排放量最高的国家。

8. 2021年排放前10的国家

```python
1.# 8.2021年排放前10的国家
2.# 过滤出2021年的数据
3.df_2021 = mdf.filter(mdf.Year == "2021")
4.# 按排放总量降序排序，并选取前10名国家
5.df8 = df_2021.orderBy(df_2021.Total.desc()).limit(10)
6.6.df8.repartition(1).write.csv('/user/xmw/df8',mode="overwrite")
7.7.# 显示查询结果
8.8.df8.show()
```

language-python

```
+------------+----+---------+---------+---------+----------+---------+---------+---------+---------+
|     Country|Year|    Total|     Coal|      Oil|       Gas|   Cement|  Flaring|    Other|PerCapita|
+------------+----+---------+---------+---------+----------+---------+---------+---------+---------+
|       China|2021|11472.369|7955.9854|  1713.34|  773.8661|852.96136| 4.677478|171.53888| 8.045741|
|         USA|2021| 5007.336|   1002.3| 2233.778| 1637.0385|41.203136| 67.68437|25.331755|14.858668|
|       India|2021|2709.6836|1802.3119|622.14813|  133.4508|149.00125| 2.771552|      0.0| 1.925088|
|      Russia|2021|1755.5474|380.22714|403.08038| 875.05176| 21.92082|58.921787|16.345491| 12.09865|
|       Japan|2021|1067.3984| 418.8205|395.35938| 221.52507|23.788868| 0.344529| 7.560075| 8.565739|
|        Iran|2021| 748.8787| 3.729952|222.96184| 466.99377|22.524908| 32.66829|      0.0| 8.517397|
|     Germany|2021|674.75354|230.22105|248.20062| 173.48405|13.233969| 1.807425| 7.806447| 8.089741|
|Saudi Arabia|2021| 672.3799|      0.0| 369.7214|  270.0651|28.688192| 3.905174|      0.0| 18.70299|
|   Indonesia|2021| 619.2775|303.15356|210.24942|  74.08785|28.637354| 3.149338|      0.0| 2.262175|
| South Korea|2021|  616.075|283.90836|171.48422|126.789734|23.704067| 0.006091| 10.18252|11.886424|
+------------+----+---------+---------+---------+----------+---------+---------+---------+---------+
```

这里统计了2021年总CO2排放量前10的国家，可以看出，排放量最高的是中国，其次是美国，印度，俄罗斯等。

9. 每个国家的历史排放总量之和，并且按照从高到低排序

```python
1.# 9. 每个国家的历史排放总量之和，并且按照从高到低排序
2.query = """
3.    SELECT Country, SUM(Total) AS TotalSum
4.    FROM usInfo
5.    GROUP BY Country
6.    ORDER BY TotalSum DESC
7."""
8.df9 = spark.sql(query)
9.# 显示查询结果
10.df9.show()
11.df9.repartition(1).write.csv('/user/xmw/df9',mode="overwrite")
```

这里统计了每个国家1750-2021年之间的CO2排放总量之和。可以看出，美国的排放量是最高的，其次是中国。

10. 计算每个国家的排放总量占比

```python
1.# 10.   计算每个国家的排放总量占比
2.# 计算全局的排放总量
3.global_total = spark.sql("SELECT SUM(Total) FROM usInfo").collect()[0][0]
4.# 使用 Spark SQL 查询每个国家的排放总量及占比
5.query = f"""
6.    SELECT Country, SUM(Total) AS TotalEmissions, SUM(Total) / {global_total} * 100 AS Percentage
7.    FROM usInfo
8.    GROUP BY Country
9.    ORDER BY TotalEmissions DESC
10."""
11.df10 = spark.sql(query)
12.df10.repartition(1).write.csv('/user/xmw/df10',mode="overwrite")
13.# 显示查询结果
```

```python
14.14.df10.show()
```

```
+--------------+------------------+------------------+
|       Country|    TotalEmissions|        Percentage|
+--------------+------------------+------------------+
|           USA|421906.85717102885| 24.90790413231432|
|         China|249352.81488609314| 14.72091743175396|
|        Russia|117547.63120305073|6.939600718078291 5|
|       Germany| 93290.50389736891| 5.507544824257291|
|United Kingdom| 78508.72024059296| 4.634880054842712|
|         Japan| 66711.08311377838|3.9383888517537566|
|         India|  57105.3886256516|3.3713022700403577|
|        France|  39105.7648973465|2.3086674855561315|
|        Canada| 34115.39523149654|2.0140535272638806|
|       Ukraine|30784.732571588596|  1.81742286146639|
|        Poland| 28196.09450864792|1.6645987307155385|
|         Italy|25065.176159068942|1.4797602698753436|
|  South Africa|21465.099949605763| 1.267224371085757|
|        Mexico| 20587.50731998682| 1.215414373892672|
|          Iran|19546.580840511248|1.1539617178884232|
|     Australia|18968.830836176872|1.1198533798137678|
```

从1750-2021年世界的维度上来看，这里统计了每个国家的历史排放量占比，可以看出美国占比大约25%，中国的排放量只有14.7%。

11. 计算煤炭、石油、天然气、水泥、燃烧和其他排放总量占比

```python
1.# 11.查询语句改写计算煤、油、气、水泥、燃烧和其他排放总量占比
2.# 使用 Spark SQL 计算每个字段的总量
```

```python
3.query = """
4.    SELECT
5.        SUM(Coal) AS CoalTotal,
6.        SUM(Oil) AS OilTotal,
7.        SUM(Gas) AS GasTotal,
8.        SUM(Cement) AS CementTotal,
9.        SUM(Flaring) AS FlaringTotal,
10.         SUM(Other) AS OtherTotal
11.    FROM usInfo
12."""
13.df11 = spark.sql(query)
14.df11.repartition(1).write.csv('/user/xmw/df11',mode="overwrite")
15.df11 = df11.collect()[0]
16.# 计算总排放量
17.total = sum(df11)
18.
19.# 计算每个字段的占比
20.emission_percentages = {
21.    field: (df11[field] / total) * 100
22.    for field in df11.asDict().keys()
23.}
24.# 打印每个字段的占比
25.for field, percentage in emission_percentages.items():
26.    print(f"{field}: {percentage:.2f}%")
```
language-python

```
CoalTotal: 47.48%
OilTotal: 33.20%
GasTotal: 15.00%
CementTotal: 2.66%
FlaringTotal: 1.09%
OtherTotal: 0.57%
```

这里计算了煤、石油、天然气、水泥、燃烧和其他排放总量的占比。可以看出，在CO2的排放量中，煤炭的排放占据了接近一半。

五、使用 Spark MLlib 组件进行数据分析

1. 相关性分析

```
1.# 将需要进行相关性分析的字段进行向量化
2.assembler = VectorAssembler(
3.    inputCols=["Year", "Total", "Coal", "Oil", "Gas", "Cement", "Flaring", "Other", "PerCapita"],
4.    outputCol="features"
5.)
6.df = assembler.transform(mdf).select("features")
7.# 计算字段之间的相关系数
8.correlation_matrix = Correlation.corr(df, "features").head()
9.# 提取相关系数矩阵
```

```
10.10.correlation_matrix_array = correlation_mat
   rix[0].toArray()
11.11.# 打印相关系数矩阵
12.12.print(correlation_matrix_array)
13.13.# 创建 DataFrame，保存到 HDFS
14.14.result_df = spark.createDataFrame(correlat
   ion_matrix_array.tolist())
15.15.result_df.write.mode("overwrite").format("
   csv").save("/user/xmw/analyst-ml-1")
```

language-python

```
[[1.         0.09183936 0.04485203 0.11847675 0.12240188 0.08124374
  0.13037357 0.0733394  0.09917805]
 [0.09183936 1.         0.92252468 0.88155979 0.76765279 0.75648464
  0.39029784 0.79028342 0.07027924]
 [0.04485203 0.92252468 1.         0.64188656 0.48249404 0.8915956
  0.1973581  0.8889763  0.0481156 ]
 [0.11847675 0.88155979 0.64188656 1.         0.89997838 0.42145797
  0.48357724 0.48515381 0.08033776]
 [0.12240188 0.76765279 0.48249404 0.89997838 1.         0.27161821
  0.589929   0.37053658 0.0835771 ]
 [0.08124374 0.75648464 0.8915956  0.42145797 0.27161821 1.
  0.09385235 0.90823967 0.01851555]
 [0.13037357 0.39029784 0.1973581  0.48357724 0.589929   0.09385235
  1.         0.16235786 0.07805068]
 [0.0733394  0.79028342 0.8889763  0.48515381 0.37053658 0.90823967
  0.16235786 1.         0.02338518]
 [0.09917805 0.07027924 0.0481156  0.08033776 0.0835771  0.01851555
  0.07805068 0.02338518 1.        ]]
```

这里分析了总排放量、煤炭、石油、天然气、水泥、燃烧和其他排放总量之间的相关性系数，后续也对这部分进行了可视化。

2.使用回归分析，预测中国2023年的总CO2排放量

```
1. 1.# 筛选特定国家的数据
```

```python
2.country = "China"  # 替换为你要预测的国家名称
3.country_df = mdf.filter(mdf.Country == country)
4.# 将字段进行向量化
5.assembler = VectorAssembler(
6.    inputCols=["Year"],
7.    outputCol="features"
8.)
9.country_df = assembler.transform(country_df)
10.# 创建 LinearRegression 模型
11.lr = LinearRegression(featuresCol="features", labelCol="Total")
12.# 拟合模型
13.model = lr.fit(country_df)
14.# 构造要预测的数据
15.prediction_data = spark.createDataFrame([(2023,)], ["Year"])
16.# 将要预测的数据进行特征向量化
17.prediction_data = assembler.transform(prediction_data)
18.# 进行预测
19.predictions = model.transform(prediction_data)
20.# 提取预测结果
21.predicted_total = predictions.select("prediction").collect()[0][0]
22.# 打印预测结果
23.print(f"Predicted Total Emission for {country} in 2023: {predicted_total}")
24.# 保存预测结果到 HDFS
25.result_df = spark.createDataFrame([(country,2023, predicted_total)], ["Country", "Year", "PredictedTotal"])
26.result_df.write.mode("overwrite").format("csv").save("/user/xmw/analyst-ml-2")
```

language-python

```
Predicted Total Emission for China in 2023: 6818.684005507705

进程已结束,退出代码0
```

这里通过中国历史上的总排放量来预测今年（2023年）的CO2排放总量，可以看到，预测得出中国2023的预测结果是6818。

3. 使用 Summarizer 函数统计信息

```python
1.# 特征选择和转换
2.assembler = VectorAssembler(
3.    inputCols=["Coal", "Oil", "Gas", "Cement", "Flaring", "Other"],
4.    outputCol="features"
5.)
6.df = assembler.transform(mdf)
7.# 计算汇总统计信息
8.summary = Summarizer.metrics("mean", "max", "min", "std")
9.# 应用 Summarizer 并计算摘要统计信息
10.summary_df = df.select(summary.summary(df.features).alias("summary"))
11.# 显示摘要统计信息
12.summary_df.show(truncate=False)
13.# 将结果保存为 Parquet 文件
14.summary_df.write.parquet("/user/xmw/analyst-ml-3")
```

language-python

```
+-------------------
```

```
xmw@031802433:~$ hdfs dfs -ls /user/xmw
Found 15 items
drwxr-xr-x   - xmw supergroup          0 2023-05-20 14:29 /user/xmw/analyst-ml-1
drwxr-xr-x   - xmw supergroup          0 2023-05-20 14:58 /user/xmw/analyst-ml-2
drwxr-xr-x   - xmw supergroup          0 2023-05-20 15:01 /user/xmw/analyst-ml-3
-rw-r--r--   1 xmw supergroup    1751806 2023-05-19 14:27 /user/xmw/dataset.csv
drwxr-xr-x   - xmw supergroup          0 2023-05-19 15:06 /user/xmw/df1
drwxr-xr-x   - xmw supergroup          0 2023-05-19 16:29 /user/xmw/df10
drwxr-xr-x   - xmw supergroup          0 2023-05-19 16:39 /user/xmw/df11
drwxr-xr-x   - xmw supergroup          0 2023-05-19 15:20 /user/xmw/df2
drwxr-xr-x   - xmw supergroup          0 2023-05-19 15:32 /user/xmw/df3
drwxr-xr-x   - xmw supergroup          0 2023-05-19 15:38 /user/xmw/df4
drwxr-xr-x   - xmw supergroup          0 2023-05-19 15:39 /user/xmw/df5
drwxr-xr-x   - xmw supergroup          0 2023-05-19 15:42 /user/xmw/df6
drwxr-xr-x   - xmw supergroup          0 2023-05-19 15:58 /user/xmw/df7
drwxr-xr-x   - xmw supergroup          0 2023-05-19 16:13 /user/xmw/df8
drwxr-xr-x   - xmw supergroup          0 2023-05-19 16:19 /user/xmw/df9
xmw@031802433:~$
```

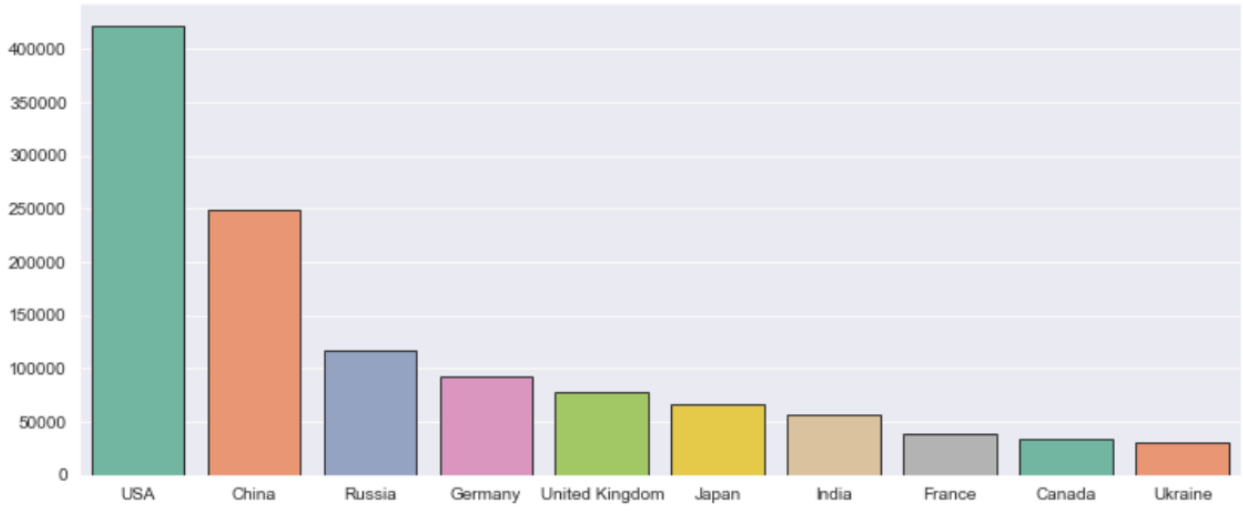
六、数据可视化

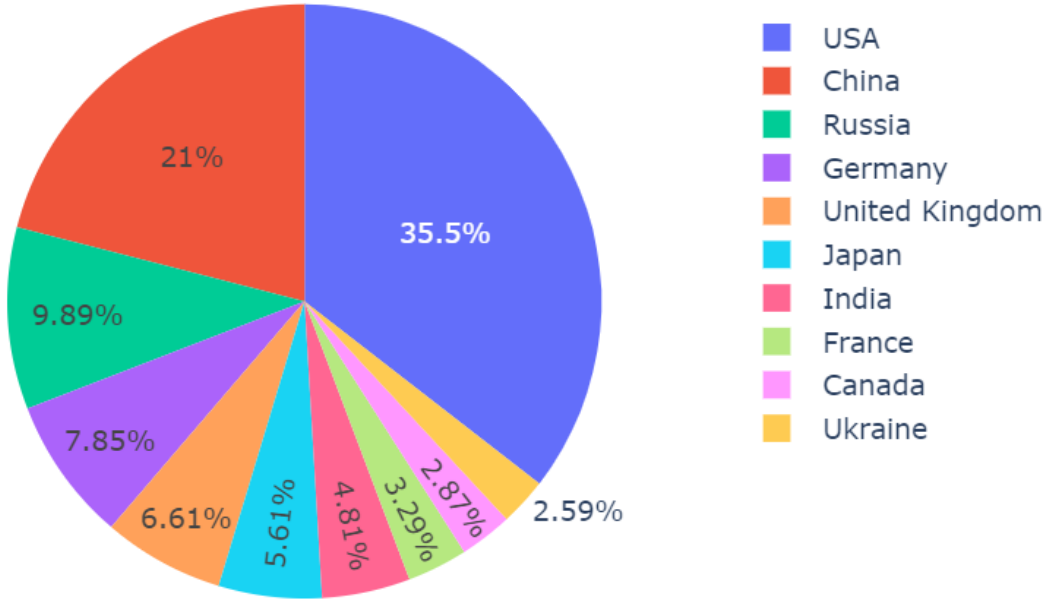
数据可视化部分采用的是jupyter notebook进行编写。

1.总排放量前10的国家

```python
1.# 1. 排放量前10的国家
2.countries = list(df.groupby('Country').sum().sort_values(by='Total',ascending=False).index)
3.values = list(df.groupby('Country').sum().sort_values(by='Total',ascending=False)['Total'])
4.
5.plt.figure(figsize=(12,5))
6.sns.set_style('darkgrid')
7.sns.barplot(x=countries[:10],y=values[:10],palette='Set2',edgecolor='.2')
```

language-python

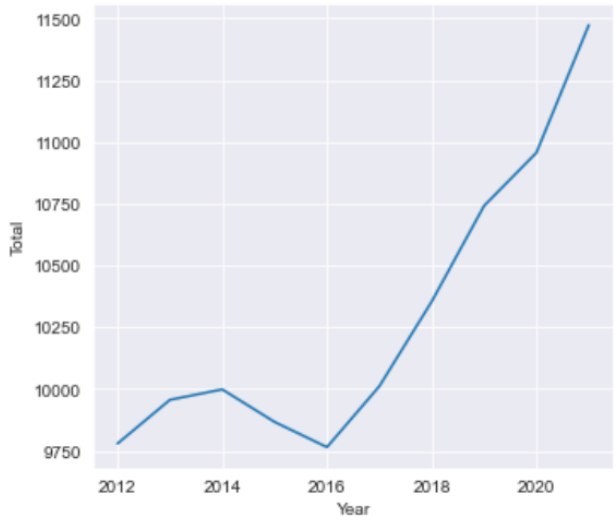


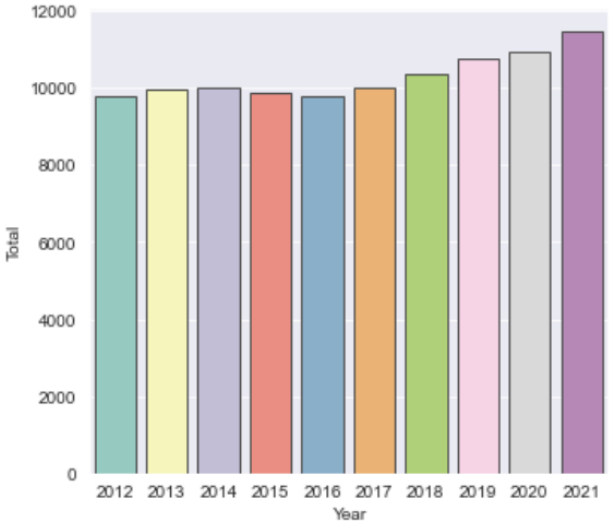


2.中国在2012-2021的总排放量趋势

```
1.# 2.中国相关数据可视化
2.china_data = df[df['Country']=='China']
3.# 前10年的变化
4.china_past_10 = china_data[-10:]
5.plt.figure(figsize=(12,5))
6.plt.subplot(121)
```

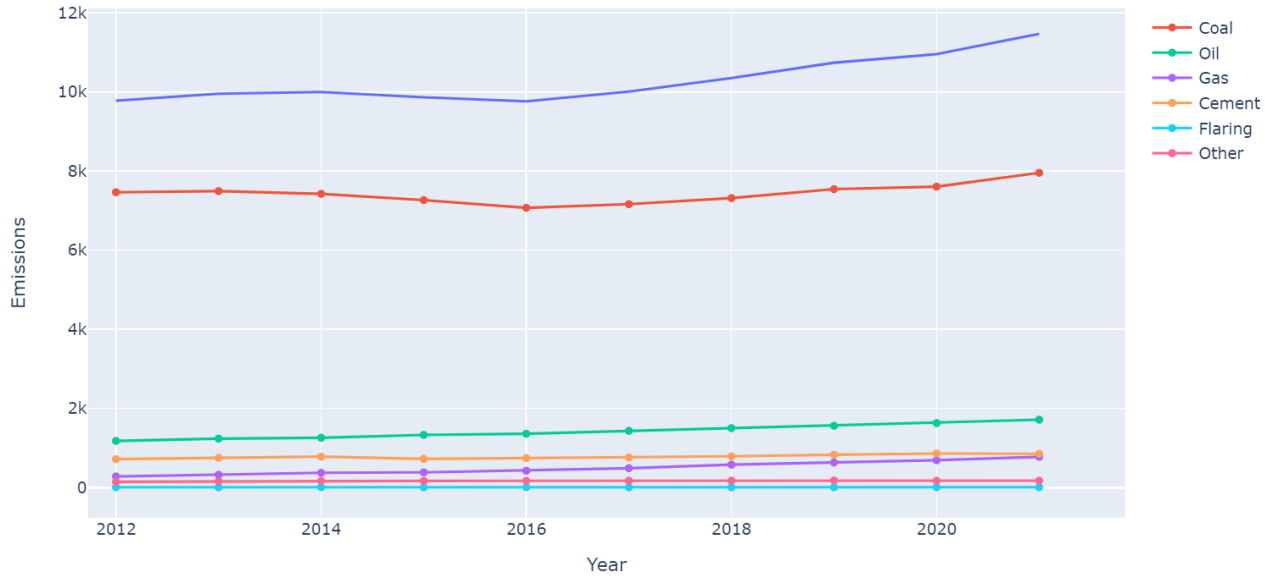
```
7. 7.sns.lineplot(x='Year',y='Total',data=china_past_10)
8. 8.plt.subplot(122)
9. 9.sns.barplot(x='Year',y='Total',data=china_past_10,palette='Set3',edgecolor='.3')
```
language-python





3.中国在2012-2021年中，各种不同物质的排放趋势

```
1. 1.#中国过去10年所有类型的物质排放趋势
2. 2.columns = ['Coal', 'Oil', 'Gas','Cement', 'Flaring', 'Other']
3. 3.plt.figure(figsize=(12,5))
4. 4.fig = px.line(y=china_past_10['Total'],x=china_past_10['Year'],labels={'x':'Year','y':'Emissions'})
5. 5.for i in columns:
6. 6.    fig.add_scatter(y=china_past_10[str(i)],x=china_past_10['Year'],name=str(i))
7. 7.fig.show()
```
language-python



4.各个字段的相关系数的可视化

```python
1.# 相关系数分析
2.plt.figure(figsize=(12,6))
3.sns.heatmap(df.corr(),cmap='YlGnBu',annot=True)
4.plt.yticks(rotation='360')
5.plt.show()
```

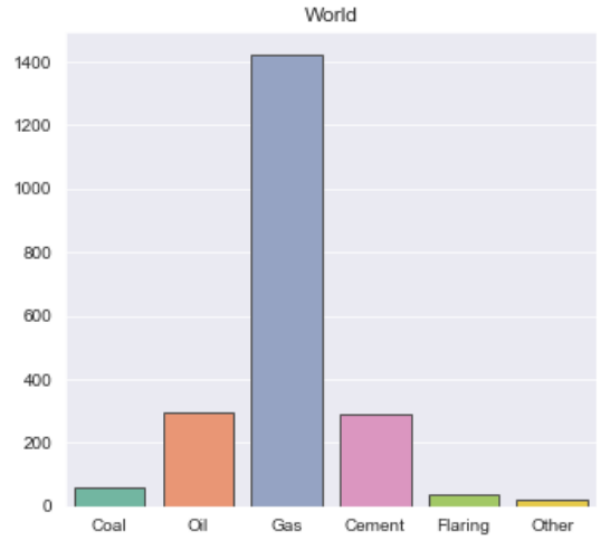
language-python

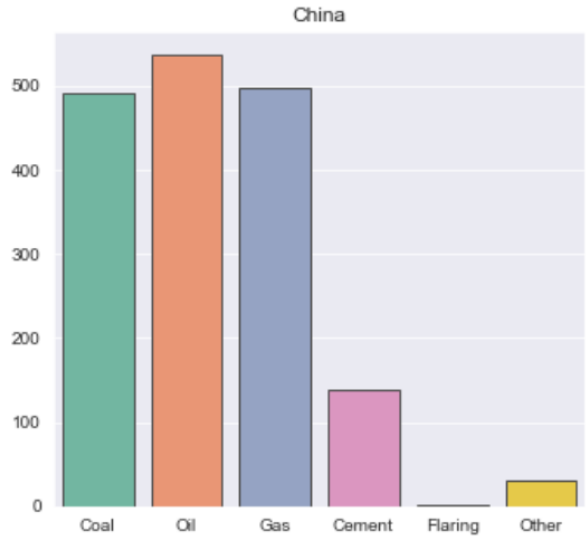


5. 在2012-2021年中，世界和中国排放增长最快的类型

```python
1.# 过去10年中，排放量增加的最快的类型
2.plt.figure(figsize=(12,5))
3.world_data_past_10 = world_data[-10:]
4.values_world = []
5.for i in columns:
6.    values_world.append(world_data_past_10.iloc[9][str(i)]-world_data_past_10.iloc[0][str(i)])
7.values_india = []
8.for i in columns:
9.    values_india.append(china_past_10.iloc[9][str(i)]-china_past_10.iloc[0][str(i)])
10.10.plt.subplot(121)
11.11.sns.barplot(x=columns,y=values_world,palette='Set2',edgecolor='.3')
12.12.plt.title('World')
13.13.plt.subplot(122)
14.14.sns.barplot(x=columns,y=values_india,palette='Set2',edgecolor='.3')
15.15.plt.title('China')
```

language-python

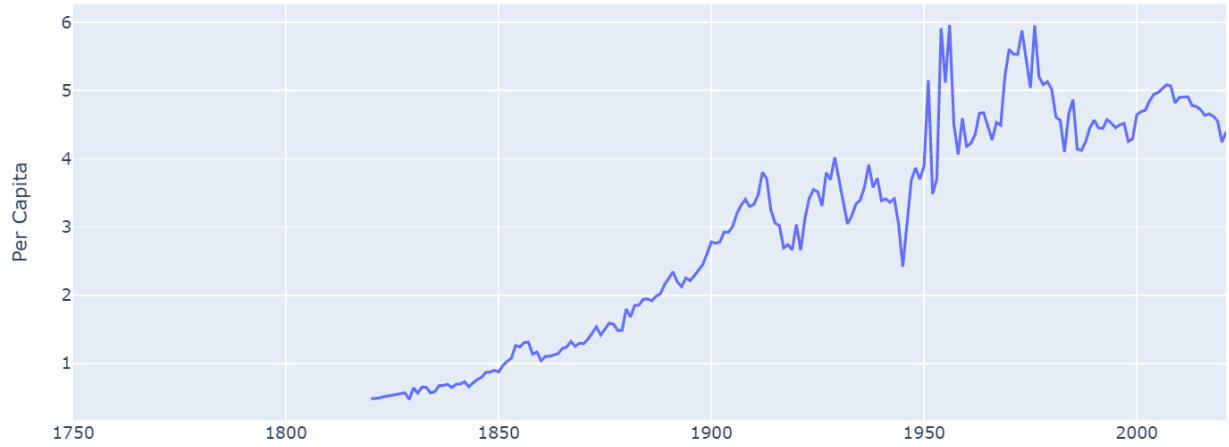




6. 历史人均排放趋势

```python
1.# 历年人均排放量趋势图
2.Year_mean_trend_pc=df.groupby(['Year'])['Per Capita'].mean().reset_index()
3.Year_mean_trend_total=df.groupby(['Year'])['Total'].mean().reset_index()
4.fig15 = px.line(Year_mean_trend_pc, x="Year", y="Per Capita")
5.fig15.update_xaxes(rangeslider_visible=True)
6.fig15.show()
```

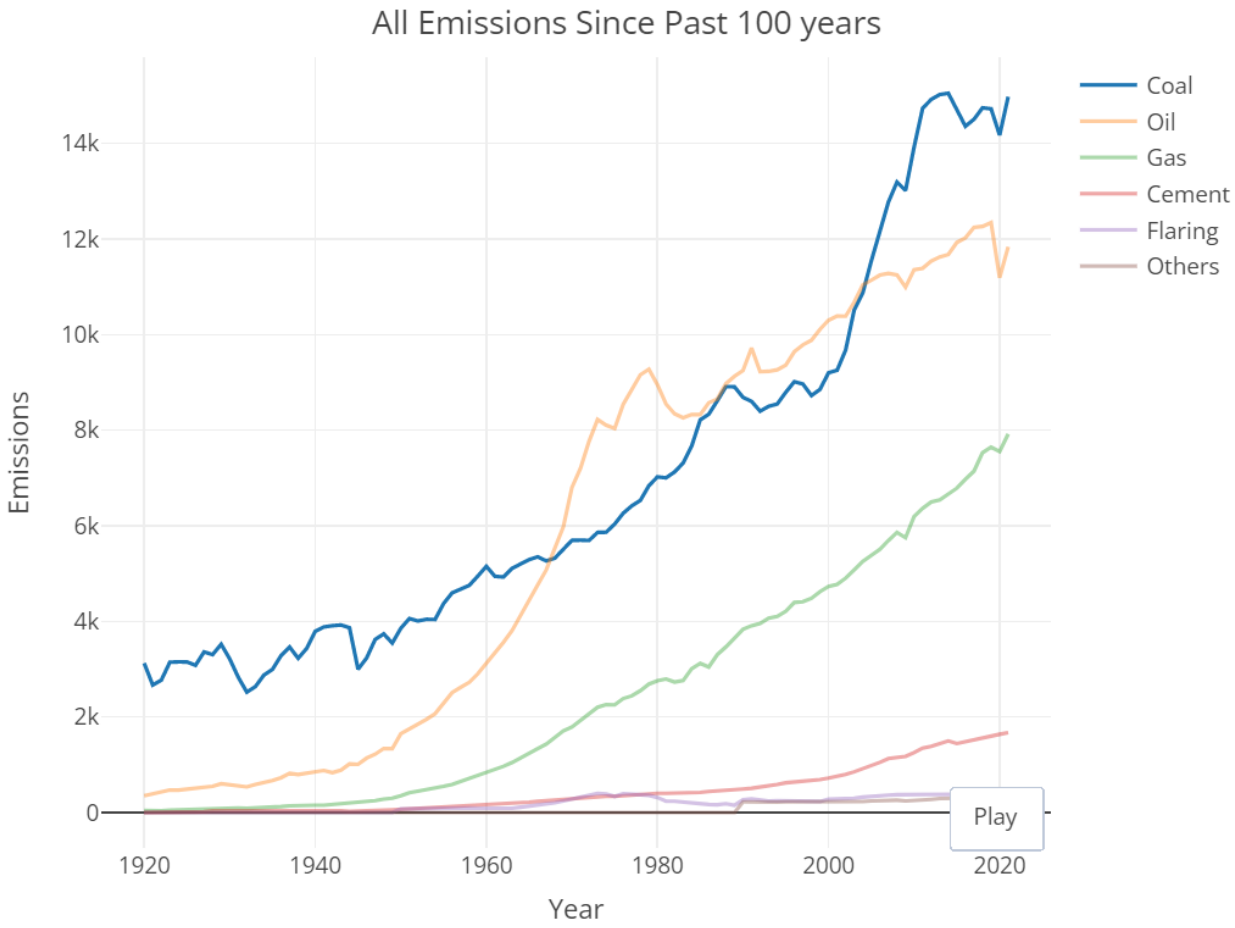
language-python



7. 全球排放趋势折线图

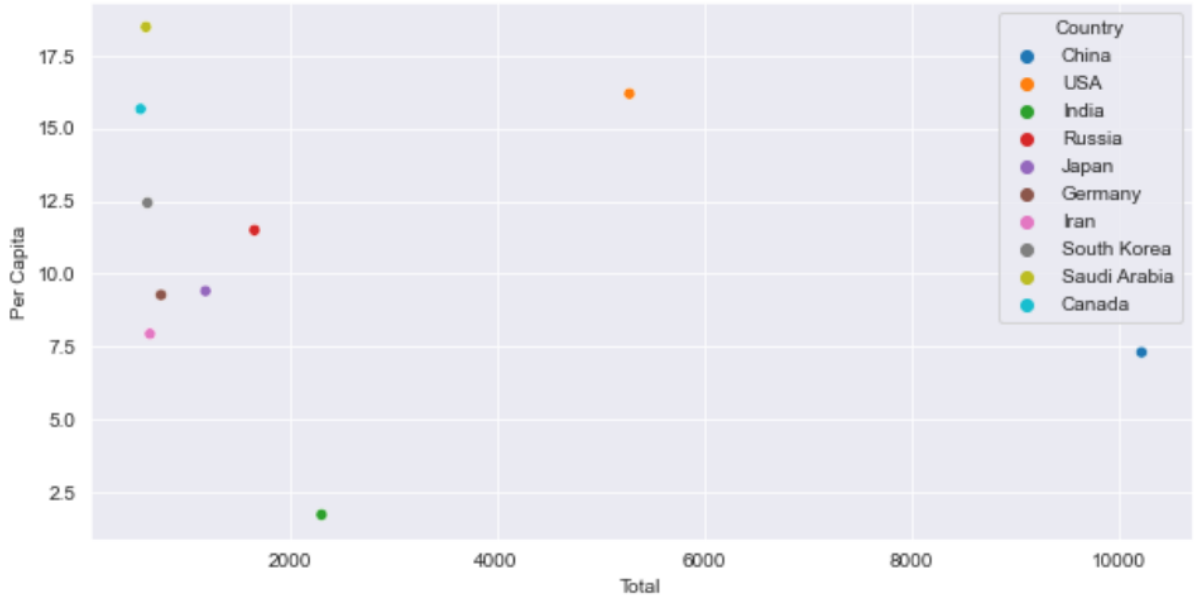
```python
1.years = data['Year'].unique()
2.categories = ['Coal', 'Oil', 'Gas', 'Cement', 'Flaring', 'Other']
3.plt.figure(figsize=(10, 6))
4.for category in categories:
5.    category_data = data.groupby('Year')[category].sum()
6.    plt.plot(years, category_data, label=category)
7.plt.title('Global CO2 Emission Trends by Category')
8.plt.xlabel('Year')
```

```
9. 9.plt.ylabel('Emission (kt)')
10. 10.plt.legend()
11. 11.plt.show()
```
language-python



8. 2011-2021年平均排放量最高的前10个国家

```
1. 1.# 过去10年平均排放量最高的国家
2. 2.plt.figure(figsize=(10,5))
3. 3.data_past_10 = df[df['Year']>=2011]
4. 4.avg_emissions = data_past_10.groupby('Country').mean().sort_values(by='Total',ascending=False)[:10].reset_index()
5. 5.sns.scatterplot(x='Total',y='Per Capita',data=avg_emissions,hue='Country')
```
language-python



9. 世界地图展示

```
1.# 世界地图展示
2.avg_data = data_past_10.groupby('Country').mean().reset_index()
3.countries = avg_data['Country'].values
4.ISO = []
5.cols = ['Total']+columns
6.for i in countries:
7.    ISO.append(data_past_10[data_past_10['Country']==str(i)]['ISO 3166-1 alpha-3'].unique()[0])
8.cols.append('Per Capita')
9.colors = ['magenta','gnbu','purp','turbo','ice','curl','oxy','haline']
10.for index,i in enumerate(cols):
11.    data = dict(type='choropleth',
12.                colorscale = str(colors[index]),
13.                locations = ISO,
14.                z = avg_data[str(i)])
15.    if i!='Per Capita':
16.        layout = dict(title = str(i) + ' Emissions By Country',
```

```
17.17.                          geo = dict( projecti
   on = {'type':'robinson'},
18.18.                                      showlakes
    = False))
19.19.     else:
20.20.         layout = dict(title = 'Per Capita
   By Country',
21.21.                          geo = dict( projecti
   on = {'type':'robinson'},
22.22.                                      showlakes
    = False))
23.23.     x = pg.Figure(data = [data],
24.24.                   layout = layout)
25.25.     po.iplot(x)
```
language-python

备注：省略世界地图效果。

总结

本次作业基于对全球CO2排放量数据的分析，大概可以得出以下结论：

排放量趋势：从分析全球排放量趋势的折线图可以看出，在过去几十年中，全球CO2排放量呈现了不断增长的趋势，尤其是在近几年，排放量增速加快。这表明全球温室气体排放问题依然严峻。

国家排放差异：不同国家之间的CO2排放量存在明显差异。少数发达国家和工业化程度较高的国家排放量较高，而发展中国家的排放量相对较低。排放量最高的国家通常是工业化水平较高的经济大国。

能源类型贡献：煤、石油和天然气等能源类型在全球CO2排放中起着重要的作用。煤炭仍然是最主要的CO2排放来源，而油和气也贡献了相当大的比例。因此，转向清洁能源和能源转型仍然是减少全球CO2排放的重要途径。

人均排放水平：人均排放量是衡量一个国家或地区碳排放水平的重要指标。发达国家和工业化程度较高的国家的人均排放量通常较高，而发展中国家的人均排放量较低。这提示我们，在减少全球CO2排放的

过程中，需要关注人均排放的差异，促进公平与可持续发展。
综上所述，全球CO2排放问题具有全球性和国家差异性，并且受到能源结构和经济发展水平的影响。在应对气候变化和降低碳排放的过程中，需要采取全球合作的措施，减少高排放国家的排放量，促进可持续发展和清洁能源的推广。